# Laboratorio 2

# Contents

**1 Introduzione** **4**

**2 Il terminale UNIX** **5**
2.1 Comandi . . . 5
2.1.1 Struttura base comandi Linux . . . 5
2.1.2 –help e man . . . 5
2.1.3 pwd . . . 5
2.1.4 ls . . . 5
2.1.5 cd . . . 6
2.1.6 mkdir . . . 6
2.1.7 touch . . . 6
2.1.8 stat . . . 7
2.1.9 cp . . . 7
2.1.10 mv . . . 7
2.1.11 rm . . . 8
2.1.12 rmdir . . . 8
2.1.13 cat . . . 8
2.1.14 less . . . 8
2.1.15 more . . . 9
2.1.16 head . . . 9
2.1.17 tail . . . 10
2.1.18 wc . . . 10
2.2 Wildcards . . . 10
2.2.1 Asterisco (*) . . . 11
2.2.2 Punto Interrogativo (?) . . . 11
2.3 Reindirizzazione . . . 11
2.3.1 Reindirizzare l'output . . . 11
2.3.2 Reindirizzare l'input . . . 11
2.3.3 Reindirizzare gli errori . . . 11
2.4 Pipe . . . 12
2.5 Virgolette . . . 12

**3 Filesystem UNIX** **13**
3.1 Path . . . 13
3.2 Marcature temporali . . . 13
3.3 File nascosti . . . 13

**4 Programmi UNIX** **14**
4.1 Flussi standard . . . 14

**5 Linguaggio C** **15**
5.1 Struttura generica . . . 15
5.2 Compilatore . . . 15
5.2.1 Warnings . . . 15
5.3 Fasi di pre-esecuzione . . . 16
5.4 Librerie . . . 16

5.5 Variabili . . . 17
5.5.1 Scoping . . . 17
5.6 printf . . . 17
5.6.1 Placeholders . . . 18
5.7 scanf . . . 18
5.8 Costanti . . . 18
5.9 Commenti . . . 19
5.10 Operatori in C . . . 19
5.11 sizeof . . . 20
5.12 If-Else . . . 20
5.13 While e do-while . . . 21
5.13.1 Break e continue . . . 21
5.14 Switch . . . 21
5.15 Casting . . . 22
5.16 Array . . . 22
5.17 Matrici . . . 22
5.18 Struct . . . 22
5.19 Enum . . . 23
5.20 typedef . . . 23
5.21 Funzioni . . . 23
5.22 Gestione della memoria . . . 24
5.22.1 Puntatori . . . 25
5.22.2 Allocazione dinamica . . . 25
5.22.3 Liberare memoria . . . 26
5.22.4 Gestione di tipi complessi . . . 26
5.22.5 Puntatore generico . . . 28
5.23 Stringhe . . . 28
5.23.1 Copia di una stringa . . . 28
5.23.2 Operazioni sulle stringe . . . 28
5.23.3 Operazioni su caratteri . . . 29
5.24 Debugging . . . 30
5.24.1 Assert . . . 30
5.24.2 GDB e Valgrind . . . 30

**6 Esercizi** **32**
6.1 Esercizio 1 . . . 32

# 1 Introduzione

In questo corso lavoreremo con programmi vicini al linguaggio macchina.
Lavoreremo inoltre con:

- UNIX
- thread
- processi
- programmazione di sistema

**Codice Teams:** r407rxv

# 2 Il terminale UNIX

Il **terminale** è una interfaccia testuale per dialogare con il sistema.
Dentro il terminale gira una **shell**, ovvero il programma che interpreta i comandi.
Il terminale mostra il **prompt**, riceve i comandi e visualizza l'output.
Il simbolo $ indica che la shell è pronta a ricevere un comando (potrebbe non essere $ a volte).

## 2.1 Comandi

### 2.1.1 Struttura base comandi Linux

Esiste una struttura base per molti comandi, ovvero questa:

comando + opzione + argomento + (opzione + argomento)

La differenza principale tra opzione e argomento è che l'opzione è relativa alla **modalità** del comando e l'argomento **su cosa lavora** il comando. L'opzione + argomento invece è relativo all'argomento. per esempio:

```
$ gcc -Wall hello.c -o hello
```

### 2.1.2 –help e man

Entrambi questi comandi aiutano a comprendere specifici comandi in modi diversi, –help è un aiuto generico mentre "man" è il manuale.

```
$ [comando] --help
$ gcc --help
$ man [comando]
$ man gcc
```

### 2.1.3 pwd

**pwd** è il comando "print working directory" e mostra la posizione della directory corrente all'interno del **file system**.

```
$ pwd
```

### 2.1.4 ls

**ls**, ovvero "list", elenca tutto il contenuto della directory corrente, directory e file.

```
$ ls
```

Può essere usata con alcuni **attributi**, aggiungibili con un "dash" (-) come in questo caso:

```
$ ls -a
```

Ecco gli attributi:

- **a**: include tutti gli elementi nascosti
- **l**: include informazioni avanzate (data, grandezza...)

### 2.1.5 cd

**cd** significa "change directory" e si sposta nella directory specificata da un path relativo o assoluto.

```
$ cd [path]
$ cd esercizi
$ cd /users/gabri/esercizi
```

Percorsi speciali:

- **Directory corrente**: specificato da un punto, ha lo stesso valore di pwd.

```
$ cd .
```

- **Directory padre**: specificato da un doppio punto.

```
$ cd ..
```

- **Home dell'utente**: specificato da una tilde, si sposta nella home dell'utente ovunque essa sia.

```
$ cd ~
```

### 2.1.6 mkdir

**mkdir** significa "make directory" e crea una directory del nome specificato nella cartella corrente.

```
$ mkdir [nomeDirectory]
$ mkdir esercizi
```

### 2.1.7 touch

Il comando **touch** nasce con l'intento di aggiornare l'ultimo accesso e l'ultima modifica di un file (marcatura temporale, timestamp).

```
$ touch [nomeFile]
$ touch log.txt
```

Se il file non esiste, lo crea.
Tale comando senza attributi modifica:

- Tempo di accesso (atime)
- Tempo di modifica (mtime)

Se desideriamo poi modificare uno solo di questi timestamp allora si può introdurre un attributo:

- **-a**: atime

- **-m**: mtime

Il comando può essere usato anche sulle directory, ma non crea automaticamente una directory se assente, bensì solo file.

### 2.1.8 stat

Il comando **stat**, ovvero "status", mostra i metadati di un file o una directory, inclusi i timestamp.

```
$ stat [nome]
$ stat hello.c
```

Ecco i dati mostrati da tale comando:

- Tipo di file
- Dimensione
- Inode
- Permessi
- Proprietario e Gruppo
- Data di ultimo accesso (Access)
- Data di ultima modifica (Modify)
- Data di ultima modifica dei metadati (Change)

### 2.1.9 cp

Il comando **cp**, ovvero "copy", copia il file passato come parametro e lo incolla nel path specificato dopo l'argomento, eventualmente rinominandolo.

```
$ cp [nomeFileDaCopiare] [Destinazione]\[NuovoNomeFile]
$ cp hello.c \esercizi\eserc.c
```

Questo esempio copia il file "hello.c", lo sposta nella sub-directory "esercizi" e lo rinomina.
Il file originale rimane al suo posto.
La rinomina della copia è omissibile.
Questo comando, con l'aggiunta di un attributo "-r" (**recursive**) può essere applicato anche alle directory.

```
$ cp -r [nomeDirectory] [Destinazione]\[NuovoNomeDirectory]
$ cp -r cartella1 \esercizi\cartella2
```

### 2.1.10 mv

Il comando **mv**, ovvero "move", sposta o/e rinomina un file:

```
$ mv [nomeVecchio] [destinazione]\[nomeNuovo]
$ mv hello.c ciao.c
```

Questo comando rinomina il file "hello.c" in "ciao.c", senza spostarlo.

### 2.1.11 rm

Il comando **rm**, ovvero "remove", elimina un file.

```
$ rm [nomeFile]
$ rm hello.c
```

La cancellazione da terminale non sposta nel cestino, è in genere definitiva.

### 2.1.12 rmdir

Il comando **rmdir**, ovvero "remove directory", elimina una directory vuota.

```
$ rmdir [nomeDirectory]
$ rmdir esercizi
```

Se vogliamo eliminare una cartella e il suo contenuto, si possono usare questi attributi:

- **-r**: "recursive", applica la cancellazione anche ai contenuti.
- **-f**: "forced", applica l'attributo recursive con forza, ignora le conferme di cancellazione.

### 2.1.13 cat

Il comando **cat**, ovvero "concatenate", scrive il contenuto del file sullo standard output.

```
$ cat [nomeFile]
$ cat hello.txt
```

Si possono stampare più file in sequenza, concatenati:

```
$ cat [nomeFile1] [nomeFile2]
$ cat hello1.txt hello2.txt
```

Il comando usato da solo diventa interattivo e legge dallo standard input, ovvero il terminale: ogni volta che si scrive qualcosa a cat, lo stamperà direttamente sul terminale.

```
$ cat
$ ciao
ciao
```

Per uscire da questa modalità bisogna premere (**Ctrl + D**).

### 2.1.14 less

Il comando **less**, opera simile al comando **cat**, ma non carica tutto il file nel terminale e per leggere più contenuto bisogna scrollare.

```
$ less [nomeFile]
$ less hello.txt
```

Si possono aprire anche più file insieme:

```
$ less [nomeFile1] [nomeFile2]
$ less file1.txt file2.txt
```

Per passare al prossimo file o al precedente si fa rispettivamente:

```
$ :n
$ :p
```

Per scorrere in avanti si può premere:

- **Spazio**: pagina successiva.
- ↓: riga successiva.
- **Invio/Enter**: riga successiva.
- **G**: fine del file.

Questo comando permette anche di scorrere indietro:

- **b**: pagina precedente
- **Freccia in su**: riga precedente
- **g**: inizio del file

e di cercare parole, vediamo un esempio (una volta attivato less):

```
$ /ciao (cerca in avanti)
$ ?ciao (cerca indietro)
```

Per scorrere le occorrenze si fa cosi:

```
$ n (occorrenza successiva)
$ N (occorrenza precedente)
```

Per uscire bisogna premere **Q**.

### 2.1.15 more

Il comando **more** funziona come il comando **less**, ma non permette di scorrere indietro, nè di cercare parole.

```
$ more [nomeFile]
$ more file.txt
```

Per uscire bisogna premere **Q**.

### 2.1.16 head

Il comando **head** permette di stampare le prime righe di un file su terminale.

```
$ head [nomeFile]
$ head hello.txt
```

Possiamo specificare quante righe stampare:

```
$ head -n [righeDaStampare] [nomeFile]
$ head -n 5 hello.txt
```

Questo esempio stampa le prime 5 righe del file "hello.txt".

### 2.1.17 tail

Il comando **tail** permette di stampare le ultime righe di un file su terminale.

```
$ tail [nomeFile]
$ tail hello.txt
```

Possiamo specificare quante righe stampare:

```
$ tail -n [righeDaStampare] [nomeFile]
$ tail -n 5 hello.txt
```

Questo esempio stampa le ultime 5 righe del file "hello.txt".

### 2.1.18 wc

Il comando **wc**, ovvero "word count", serve per contare delle specifiche informazioni in un file.

```
$ wc [nomeFile]
$ wc file.txt
```

Il comando restituisce 3 valori in questo ordine:

- Numero di righe
- Numero di parole
- Numero di byte

Il comando può essere eseguito con un filtro passato come attributo:

- **-l** per le righe
- **-w** per le parole
- **-c** per i byte

## 2.2 Wildcards

Le wildcard permettono di mantenere un aspetto generico di alcuni dati per lavorare su più entità in contemporanea.
Le wildcard quindi funzionano da veri e propri **placeholder**. Questa è una funzionalità della shell, la quale **espande** le wildcard una volta che gli arrivano: sostituisce le wildcard e genera un elenco di tutte le unità che soddisfano la wildcard.
Per esempio:

```
$ ls *.c
"espansione shell"
$ ls file1.c file2.c file3.c
```

Ai comandi non viene mai passata la shell, bensì gli argomenti precisi su cui lavorare.

### 2.2.1 Asterisco (*)

L'asterisco (*) usato come wildcard permette di specificare una **stringa qualsiasi** di lunghezza qualsiasi, per esempio:

```
$ ls *.c
```

Questo comando stampa tutti i file con estensione ".c", ovvero i file con nome qualsiasi che hanno estensione ".c".

### 2.2.2 Punto Interrogativo (?)

Il punto interrogativo (?) usato come wildcard indica un **carattere** qualsiasi.

```
$ ls prova?.c
```

Questo comando stampa tutti i file con prefisso "prova", suffisso ".c" e un carattere qualsiasi che divide suffisso e prefisso specificati, in questo caso utile per stampare tutte le prove / test possibilmente nominati "prova1", "prova2" e cosi via.

## 2.3 Reindirizzazione

### 2.3.1 Reindirizzare l'output

Si può reindirizzare lo **standard output** di un comando in questo modo:

```
$ [comando] > [destinazione]
$ ls > lsLog.txt
```

Questo comando scrive in "lsLog.txt" quello che sarebe dovuto essere stampato sul terminale dal comando "ls".

**Attenzione:** questo tipo di reindirizzazione **sovrascrive** ogni volta il file intero, se si desidera eseguire un'aggiunta in fondo al file senza perdere i dati precedenti allora bisogna usare l'**append**:

```
$ [comando] >> [destinazione]
$ ls >> lsLog.txt
```

### 2.3.2 Reindirizzare l'input

Come l'output, si può reindirizzare lo **standard input**:

```
$ [destinazione] < [sorgente]
$ sort < dati.txt
```

In questo esempio, si stampa in ordine alfabetico il contenuto di "dati.txt"

### 2.3.3 Reindirizzare gli errori

Anche il flusso **standard error** può essere reindirizzato:

```
$ [comando] 2> [destinazione]
$ ls 2> lsLog.txt
```

## 2.4 Pipe

La **pipe** collega più comandi in questo modo:

stdout del primo comando → stdin del secondo comando

Ecco il template e un esempio:

```
$ [comando1] | [comando2]
$ ls | less
```

In questo esempio viene eseguito "ls" e viene poi stampato secondo il comando "less".
La pipe evita di creare file intermedi che sarebbero necessari per realizzare il solito effetto.

## 2.5 Virgolette

Per evitare che non ci sia ambiguità negli argomenti, dato che la shell utilizza gli spazi per separare argomenti, comandi e opzioni, bisogna utilizzare le **virgolette**:

```
$ mkdir "nuova cartellozza"
```

# 3 Filesystem UNIX

Il **filesystem** è un albero che ha come radice la **root**, indicata da un "/"

## 3.1 Path

Il filesystem localizza ogni file e directory tramite un **path**, il quale può essere:

- **Assoluto:** parte dalla root.
- **Relativo:** parte dalla directory corrente.

## 3.2 Marcature temporali

tenere traccia del tempo in UNIX è importante: i timestamp sono spesso utilizzati per prendere decisioni importanti, come la ricompilazione di un file con un timestamp vecchio. In UNIX ogni file ha 3 marcature temporali:

- **atime**: access time, ultimo accesso.
- **mtime**: modification time, ultima modifica del contenuto.
- **ctime**: change time, ultima modifica dei metadati.

## 3.3 File nascosti

I **file nascosti** sono i file il cui nome solitamente inizia con il punto.

.txt .config .bashrc

# 4 Programmi UNIX

## 4.1 Flussi standard

Ogni programma ha 3 flussi standard:

- **standard input**
- **standard output**
- **standard error**

Questi flussi possono essere reindirizzati altrove dal terminale.

# 5 Linguaggio C

Il linguaggio **C** è un linguaggio compilato, pertanto, prima di eseguire un file bisogna convertirlo in binario.
C è un linguaggio **strongly typed**, c'è un forte controllo dei tipi.
Necessita ";" (semicolon) alla fine di ogni istruzione.
Esistono diversi standard C: C89, C95, C99, C11, C17, C23.

## 5.1 Struttura generica

Un programma C solitamente è composto da questi elementi in questo ordine:

1. Direttive pre-processore
2. Dichiarazioni globali
3. Funzioni (incluso main)

**ATTENZIONE:** un programma C può avere **un solo main**.

## 5.2 Compilatore

Il compilatore più conosciuto e usato per C è **gcc**, ovvero "GNU C Compiler", anche se adesso viene definito come "GNU Compiler Collection" perche diventato un progetto più ampio che include anche altri compilatori come quello di Fortran e C++.

```
gcc hello.c -o hello
```

In questo caso, il comando:

- compila il file "hello.c"
- rinomina l'eseguibile come "hello"

Ogni programma eseguito ritorna un risultato per determinare se è terminato con successo o meno, si ottiene cosi:

```
$ gcc hello.c
$ hello.c
$ echo $?
0
```

### 5.2.1 Warnings

I **warnings** sono importanti e non vanno ignorati, per compilare e stamparli tutti si fa cosi:

```
$ gcc -Wall hello.c
```

Ogni warning contiene un messaggio e la location (riga e carattere) di esso.

## 5.3 Fasi di pre-esecuzione

Ecco cosa succede dall'avvio del programma:

- **Preprocessing**: vengono eseguite le direttive pre-processore.
- **Compilazione**: il sorgente .c viene trasformato in codice oggetto .o.
- **Linking**: i file oggetto vengono collegati in un unico eseguibile.
- **Caricamento**: l'eseguibile viene caricato in memoria per essere eseguito, viene diviso in 4 **zone** diverse:
  - **Codice**: contiene le istruzioni e le funzioni.
  - **Dati statici**: variabili globali, dati che rimangono per tutta la durata dell'esecuzione.
  - **Heap**: zona dinamica, mantiene i dati temporanei come variabili ed oggetti.
  - **Stack**: zona dinamica, quando una funzione entra in esecuzione viene generato un **record di attivazione**, il quale contiene varie informazioni legate a tale chiamata. Uno stack viene eliminato al terminare di un blocco di funzione.

Vediamo come svolgere queste fasi manualmente invece che compilare tutto insieme direttamente:

- Preprocessing: con "-E" ci si ferma alle direttive di preprocessing, "-o" invece salva l'output in un file secondario.

```
$ gcc -E [nomeSorgente] -o [nuovoNome]
$ gcc -E codice.c -o codiceP.c

```

- Compilazione: si accettano solo file ".o", qui con "-c" si compila ma ci si ferma prima del Linking.

```
$ gcc -c [nomeFile] -o [nuovoNome]
$ gcc -c codiceP.c -o codice.o

```

- Linking:

```
$ gcc [nomeFile] -o [nuovoNome]
$ gcc codice.o -o eseguibile.out

```

## 5.4 Librerie

Nei programmi in C è possibile utilizzare delle **direttive di preprocessazione** (o del preprocessore) che permettono di importare funzioni e strumenti utili allo svolgimento del programma:

```
#include <stdio.h>
```

## 5.5 Variabili

In C possiamo usare le **variabili**. Sono celle di memoria a cui si può associare un valore di un tipo specifico, assegnato al momento della **dichiarazione**.

```
1     int x; //Dichiarazione
2     x = 32; //Assegnamento
```

La dichiarazioni servono al compilatore per capire quanta memoria riservare.
Le variabili possono essere di molti tipi:

- `char`: carattere, 1 byte
- `short`: numero intero, almeno 2 byte (piu corto di int comunque)
- `int`: numero intero, almeno 2 byte
- `long`: numero intero, almeno 4 byte
- `long long`: numero intero, almeno 8 byte
- `float`: numero reale a precisione singola, almeno 4 byte
- `double`: numero reale a precisione doppia, almeno 8 byte
- `long double`: numero reale a precisione estesa, almeno 10 byte
- `void`: assenza di tipo, 0 byte

La presenza della parola "almeno" consiste nel fatto che non si sa mai quanto veramente sia grande una variabile e in quanti bit consista, però alcune parole chiave garantiscono una lunghezza minima.
I char possono essere assegnati a numeri corrispondenti a caratteri ASCII.
Come si può vedere non esiste nessun tipo **string**, bensì sono rappresentate come array di char:

```
1     char y [8] = "Gabriele";
```

L'insieme dei valori delle variabili di un programma C in un determinato momento rappresenta lo **stato** del programma.

### 5.5.1 Scoping

Quando dichiariamo una variabile dobbiamo sapere quando inizia ad essere visibile e quando non lo è più, per questo si deve studiare lo **scoping**. Ci sono due tipi di scoping:

- **Scoping globale**: tutte le variabili dichiarate fuori dalle funzioni (anche main)
- **Scoping locale**: le variabili definite dentro un blocco sono visibili solo nel blocco stesso.

## 5.6 printf

Questo è un comando base per stampare qualcosa su console:

```
1     printf("Hello World");
```

### 5.6.1 Placeholders

Per stampare le variabili bisogna utilizzare i **placeholders**:

```
int x = 10;
printf("%d",x)
```

Bisogna scegliere il placeholder corretto per riuscire a stampare una variabile; la scelta va in base al tipo:

- **%d**: numeri interi (int)
- **%i**: numeri interi (int)
- **%f**: numeri decimali (double)
  - Per stampare uno specifico numero di cifre decimali si aggiunge un "." dopo "%" e poi si specifica il numero:

    ```
    printf(%.2f,x)

    ```

    Questa stampa mostrerà per esempio solo 2 cifre decimali. Usato sugli altri placeholder questo provoca l'ingrandimento di padding a destra (numero negativo) o sinistra (numero positivo) di ciò che viene stampato.
- **%c**: caratteri (char)
- **%s**: stringa
- **%u**: numeri interi senza segno (unsigned int)
- **%x**: numero intero in esadecimale
- **%p**: indirizzo di memoria, puntatore
- **%z**: tipo di sizeof (size_t)

## 5.7 scanf

Questo comando serve per l'input da tastiera:

```
scanf("Numero: %d",&intero)
```

In questo caso, legge un intero e lo salva nella variabile *intero*.
La & serve per passare la variabile come indirizzo e non valore (per array non si usa, sono gia riferimenti).

## 5.8 Costanti

In C non esistono le variabili costanti, possono essere sempre modificate!

## 5.9 Commenti

I commenti in C possono essere inseriti in vari modi:

- Commenti su una riga: aperti da "//", non vengono chiusi
- Commenti su più righe: delimitati da "/*" e da "*/".

I commenti sono fondamentali e spariscono al momento di compilazione.
Sarebbe preferibile utilizzare sempre i commenti con "/*" perchè quelli con "//" sono più idonei per C++.

## 5.10 Operatori in C

### Operatori aritmetici

- `+`: addizione
- `-`: sottrazione
- `*`: moltiplicazione
- `/`: divisione
- `%`: resto della divisione (modulo)

Bisogna stare attenti perchè alcune operazioni possono ritornare errori pratici di **overflow** o **underflow**. **Operatori logici**

- `&&`: AND logico
- `||`: OR logico
- `!`: NOT logico

### Operatori di assegnamento

- `=`: assegnamento
- `+=`: somma e assegnamento
- `-=`: sottrazione e assegnamento
- `*=`: moltiplicazione e assegnamento
- `/=`: divisione e assegnamento
- `%=`: modulo e assegnamento

L'assegnamento avviene da destra verso sinistra:

```
1     a = b = c; ---> a = (b = c)
```

### Operatori di incremento e decremento

- `++`: incremento di 1

- `--`: decremento di 1
- `++x`: pre-incremento (incrementa e poi assegna)
- `x++`: post-incremento (assegna e poi incrementa)
- `--x`: pre-decremento (decrementa e poi assegna)
- `x--`: post-decremento (assegna e poi decrementa)

**Operatori di confronto**

- `==`: uguale
- `!=`: diverso
- `>`: maggiore
- `<`: minore
- `>=`: maggiore o uguale
- `<=`: minore o uguale

## 5.11 sizeof

La keyword **sizeof** restituisce quanti byte occupa in memoria un tipo di dato o una variabile:

```
1     int x = 10;
2     printf("%zu\n",sizeof(x))
```

Il placeholder utilizzato (z) è specificato in una sottosezione precedente.

## 5.12 If-Else

Per prendere decisioni si usano i costrutti **if-else**:

```
1     if (condizione) {
2         comando;
3         ...
4     } else {
5         comando;
6         ...
7     }
```

La condizione deve essere paragonabile con un valore **booleano** per determinare la decisione.
Questi costrutti si possono annidare e addirittura si può omettere l'else.

## 5.13 While e do-while

Questo costrutto permette di eseguire più volte le stesse istruzioni fino ad un certo **break point**, ovvero fino a che una condizione non ferma il while.

```
while (condizione) {
    comando;
    ...
}
```

C'è un'opzione alternativa che garantisce una prima esecuzione e avvia il ciclo solo se la condizione è soddisfatta:

```
do {
    comando;
    ...
} while (condizione)
```

### 5.13.1 Break e continue

Il **break** serve per interrompere un ciclo:

```
while(condizione) {
    comando;
    if(condizione) {
        break;
    }
}
```

Il **continue** invece interrompe solo un iterazione e passa alla prossima:

```
while(condizione) {
    comando;
    if(condizione) {
        continue;
    }
}
```

## 5.14 Switch

Lo **switch** è un costrutto decisionale alternativo a if-else per scegliere tra più di 2 scelte:

```
switch (espressione) {
    case valoreCostante1:
        comando1;
        comando2;
        break;
    case valoreCostante2:
        comando3;
        comando4;
        break;
    default:
        break;
}
```

Il break è necessario per evitare errori.

## 5.15 Casting

Il **casting** permette di convertire esplicitamente un valore da un tipo a un altro tramite la sintassi "(tipo) espressione".

```
int a = 5;
int b = 2;
float risultato = (float)a / b;
```

In questo caso, il casting di **a** a **float** permette di ottenere il risultato decimale **2.5**, evitando la divisione intera.

## 5.16 Array

L'**array** è la prima struttura non banale che vediamo, corrisponde ad una **sequenza** di valori dello stesso tipo, in memoria corrisponde ad una sequenza di celle.
L'array è una struttura dati di lunghezza fissa (non può essere modificata) ed è ad **accesso diretto** tramite gli **indici (index)**, i quali partono da "0".
**Definizione array:**

```
    [tipo] [nomeArray][lunghezza]
    double reali[10]
```

**Accesso ad un elemento:**

```
    [nomeArray][index]
    reali[0]
```

Un array è passato per riferimento alle funzioni, in quanto il suo nominativo è proprio un indirizzo al primo elemento. Se si modifica l'array in una funzione quindi, viene modificato anche al di fuori.

## 5.17 Matrici

Le **matrici** non sono altro che **array di array**:

```
    int mat[10][10];
```

Questo crea una matrice $10x10$ e la metodologia di accesso è la medesima degli array.

## 5.18 Struct

Lo **struct** permette di definire **tipi custom**, strutture dati non omogenee (non con membri tutti dello stesso tipo).
Crea nuovi tipi, non variabile. Utilizzato per creare gli oggetti.
**Definizione oggetto tramite struct:**

```
    struct Studente {
        char nome[10];
        int eta;
    }
```

**Accesso a un membro:**

```
[nomeStruct].[membro]
studente.[eta]
```

L'ordine dei membri influisce sull'ordine in memoria.

## 5.19 Enum

**Enum** definisce un tipo enumerativo, rende il codice più autodocumentante.
Ogni identificatore è una variabile costante intera che di default parte da 0 e aumenta di 1 per ogni chiave.
I valori assegnati agli identificatori sono custom.

```
enum giorni {lunedi, martedi, mercoledi, giovedi, venerdi, sabato, domenica};
enum giorni giorno = "lunedi";
enum giorni {lunedi = 1, martedi, mercoledi = 4, giovedi, venerdi, sabato, domenica};
```

## 5.20 typedef

**typedef** serve per creare un **alias** per un tipo già esistente:

```
typedef [tipo] [nuovoNome]
typedef int Intero
```

Attenzione, **struct** non necessita typedef:

```
struct Persona {
    ...
}
```

## 5.21 Funzioni

Le **funzioni** in C sono astrazioni di blocchi di codice con un **nome**.
Ogni programma è costituito da uno o più file che contengono funzioni.
Le funzioni sono tutte **globali**.
La prima ad essere eseguita in un programma è sempre **main**.
Una funzione è fatta da 3 componenti:

- **Tipo**: specifica il tipo del risultato.
- **Nome**: serve da identificativo per richiamare la funzione.
- **Parametri formali**: generici.

Una funzione ha:

- Una **dichiarazione** (prima del main), realizzabile solo con la **firma** della funzione (tipo, nome e tipo dei parametri)

- Una **definizione** (dopo il main), realizzabile ripetendo la firma e includendo il blocco di codice da svolgere della funzione.
- Eventuali **chiamate** (nel main), quando la funzione deve essere eseguita specificando solo nome e **parametri attuali**.

```c
#include <stdio.h>

int somma(int, int);

int main(void) {
    int a = 2;
    int b = 3;
    int res;

    res = somma(2,3);
    printf("%d",res);
}

int somma(int a, int b) {
    return a + b;
}
```

Se desideriamo, possiamo omettere di ritornare un valore con una funzione tramite il tipo **void**.

```c
void somma(int a, int b) {
    int c = a + b;
    printf("%d",c);
    return;
}
```

Addirittura, una funzione può non necessitare parametri:

```c
void stampaCiao() {
    printf("Ciao :D");
    return;
}
```

Il tipo di ritorno deve essere compatibile, ma non precisamente uguale, con il tipo specificato nella firma. Questo perchè si possono utilizzare cast impliciti.
In C tutti i parametri sono passati per **valore**.
La chiamata è un'espressione e corrisponde al valore ritornato, il quale può essere riusato in altre funzioni come quella di stampa.

## 5.22 Gestione della memoria

Un qualsiasi dato/istruzione che sia in memoria ha:

- Un valore.
- Un indirizzo di memoria.

### 5.22.1 Puntatori

Un **puntatore** è una variabile che contiene un indirizzo di memoria:

```
1     int a = 3;
2     int *addr = &a;
```

Come valore ha un indirizzo di memoria, ma ha a sua volta un suo indirizzo di memoria.
Il tipo del puntatore corrisponde al tipo del valore salvato nell'indirizzo di memoria specificato dal puntatore.
Deveono sempre essere inizializzati a **null** se non ad un indirizzo di una variabile.
I puntatori possono essere dichiarati in sequenza:

```
1     int a = 3;
2     int *addr = &a;
3     int **a = &addr;
```

Per leggere i dati annessi ad un puntatore si fa cosi:

```
1     int a = 3;
2     int *addr = &a;
3
4     printf("%p\n",(void *)addr); /*stampa indirizzo della variabile*/
5     printf("%d\n",*addr); /*stampa il valore della variabile puntata*/
6     printf("%p\n",&p); /*stampa l'indirizzo del puntatore*/
7
8     *addr = 25; /*modifica il valore della variabile puntata*/
```

**Attenzione:** se si tenta di stampare il valore della variabile puntata (con *addr) e il puntatore non è stato inizializzato oppure ha valore NULL allora avverrà un **Runtime Error**.
I puntatori sono rappresentati come numeri interi, ma C non li tratta come tali: si possono usare in espressioni con altri puntatori, ma non con variabili intere.
Per poter cambiare il valore di una variabile nell'ambiente del chiamante di una funzione dall'interno di una funzione si possono usare proprio i puntatori, passandoli come parametro.

### 5.22.2 Allocazione dinamica

Ricordiamo come i vari tipi di dati vengono salvati in varie zone:

- **Stack:** record di attivazione, variabili locali
- **Heap:** dati dinamici (runtime), frammentati, memoria allocata dinamicamente.

Per eseguire un'allocazione dinamica della memoria, si utilizzano i **puntatori**.
Ogni puntatore che punta ad un allocazione dinamica può essere visto come un array, solo che si evita di stabilire la memoria allocata prima del runtime.
Una prima opzione è usare **malloc**.

```
1     malloc(grandezzaArrayAssociato * sizeof(tipo));
2
3     int *aPtr;
4     aPtr = (*int)malloc(sizeof(int));
```

```
int arr[5];
/*con malloc sarebbe:*/
int *p = (int*)malloc(5 * sizeof(int));
```

Si accede nel solito modo in cui si accede un array comune, ovvero con gli indici ad accesso diretto.
Un altro modo è usare **calloc**, il quale inizializza tutti gli elementi a 0:

```
calloc(grandezzaArrayAssociato,sizeof(tipo));

int *aPtr;
aPtr = (*int)calloc(1,sizeof(int));
```

### 5.22.3 Liberare memoria

Per liberare la memoria allocata si usa la keyword **free** (usabile solo sui puntatori).

```
free(aPtr);
```

Una volta che qualcosa non ci serve più, bisogna eliminarlo.

**Errori tipici:**

- Liberare uno spazio già vuoto (genera errore).
- Consumare memoria non liberandola e perdendo gli indirizzi dei puntatori (**memory leak**).
- Liberare memoria allocata in modo statico.
- Utilizzare un puntatore dopo aver liberato la memoria (può essere stata anche un altrà entità attraverso un puntatore secondario).

### 5.22.4 Gestione di tipi complessi

Vediamo un puntatore ad una struct:

```
typedef struct {
    int elements[N];
    int n;
} Stack;
Stack* s = (Stack*)malloc(sizeof(Stack));
/*Per richiamare un campo si fa cosi:*/
s -> n;
```

I puntatori possono essere usati anche per **liste concatenate**:

```
/*Definizione nodo*/
typedef struct {
    int val;
    /*Variabile di tipo puntatore a struct n*/
```

```
        struct n * next;
    } Node;
    Node* head = NULL; /*Lista vuota*/

    Node elem;
    elem.val = 10;
    elem.next = NULL;

    head = &elem; /*Lista con 1 elemento*/

    Node elem1;
    elem1.val = 20;
    elem1.next = NULL;

    elem.next = &elem1; /*Aggiungo in coda*/

    Node elem2;
    elem2.val = 5;
    elem2.next = head; /*Aggiungo in testa*/
    head = &elem2;

    /*Iterazione su lista*/
    Node* l=head;
    while(l!=NULL) {
        printf("%d ", l->val);
        l = l -> next;
    }
```

I puntatori addirittura permettono di gestire le **funzioni** come se fossero variabili:

```
    int sum(int,int);
    int mul(int,int);

    int main() {
        /*f = un puntatore ad una funzione che prende
        due variabili intere e restituisce un intero*/
        int (*f)(int,int);

        f = sum;
        printf("3+6=%d\n",(*f)(3,6));

        f = mul;
        printf("3*6=%d\n",(*f)(3,6));
    }

    int sum (int x, int y) {
        return x + y;
    }

    int mul (int x, int y) {
        return x * y;
    }
```

### 5.22.5 Puntatore generico

Per realizzare un puntatore che accetta un qualsiasi tipo, si fa cosi:

```
void* p;
```

## 5.23 Stringhe

C non ha un tipo specifico per le stringhe, si realizzano con array di **char**:

```
char stringa[] = "hello";
```

Per modificare o accedere ad un carattere, si tratta la stringa come l'array che è:

```
stringa[2] = 'L';
```

### 5.23.1 Copia di una stringa

```
char s1[] = "ciao";
char s2[5];

int i=0;
while(s1[i]!='\0') {
    s2[i] = s1[i];
    i++;
}
s2[i] = '\0';
```

### 5.23.2 Operazioni sulle stringe

La libreria **string.h** offre:

```
/*Lunghezza della stringa*/
strlen(s);

/*Copia src stringa in dest, restituisce dest*/
strcpy(dest, src);

/*Confronto lessicografico, restituisce un valore:
< 0 se str1 < str2,
0 se str1 == str2,
> 0 se str1 > str2 */
strcmp(str1, str2);

/*Concatena una copia di src a dest*/
strcat(dest, src)

/*Tokenizza una stringa, delimitatore = delim*/
strtok(str, delim)
```

Vediamo in particolare un esempio di **strtok**:

```c
    char s[100];
    char* tok;

    /*Leggi la stringa fino a quando non vai a capo*/
    scanf("%[\n]",s);

    /*Suddivide la stringa in due parti, prima e dopo la prima virgola (primo elemento)*/
    tok = strtok(s,",");

    /*Stampa il primo elemento*/
    printf("%s\n",tok);

    /*Iterazione su tutti gli elementi separati da delim*/
    /*strok(NULL,",") permette di prendere il prossimo elemento*/
    /*In particolare, continua dall'ultimo stato dell'ultima stringa analizzata, ovvero s*/
    while((tok = strtok(NULL,","))!=NULL) {
        printf("%s\n",tok);
    }
```

### 5.23.3 Operazioni su caratteri

La libreria **ctype.h** offre:

```c
    /*Controlla se il carattere e' una lettera*/
    isalpha(c);

    /*Controlla se la lettera e' maiuscola*/
    isupper(c);

    /*Controlla se la lettera e' minuscola*/
    islower(c);

    /*Controlla se il carattere e' numerico*/
    isdigit(c);

    /*Controlla se il carattere e' una cifra o un numero (alfanumerico)*/
    isalnum(c);

    /*Controlla se il caratter e' uno spazio*/
    isspace(c);

    /*Converte il carattere in maiuscolo se possibile, altrimenti lo ritorna invariato*/
    toupper(c);

    /*Converte il carattere in minuscolo se possibile, altrimenti lo ritorna invariato*/
    tolower(c);
```

## 5.24 Debugging

Un primo approccio al debugging consiste nella stampa dei valori delle variabili. stdout è **buffered**: la stringa non viene necessariamente mostrata subito, ma può essere accumulata nel buffer.

Il metodo più classico è aggiungere stampe di variabili in punti cruciali del programma in modo da verificare l'esattezza dei valori.

In alternativa, is può usare lo stream **stderr** (standard error).
La funzione seguente permette di scegliere esplicitamente lo stream sul quale scrivere:

```
fprintf(stderr, "Messaggio");
```

Possiamo scegliere anche un file se vogliamo come stream alternativo.

### 5.24.1 Assert

In fase di sviluppo possiamo utilizzare la **macro assert** per assicurarci che lo stato del programma sia corretto.
**Libreria:** assert.h
Assert riceve un'espressione intera, se è vera, l'esecuzione continua, altrimenti il programma stampa un messaggio di errore su stderr e termina subito l'esecuzione.
Per disabilitare assert possiamo definire la macro NDEBUG.
Ecco un esempio dell'utilizzo di assert:

```
#include <stdio.h>
#include <assert.h>

int main() {
    int x;
    printf("Inserisci un numero positivo");
    scanf("%d",&x);
    assert(x>0);
    printf("Hai inserito %d\n", x);
}
```

### 5.24.2 GDB e Valgrind

Il debugging tramite la stampa dello stato delle variabili è poco efficiente e poco scalabile.
Possiamo utilizzare tool esistenti:

- **gdb**: permette di eseguire il programma passo per passo e verificare i valori delle variabili.
- **valgrind**: permette di analizzare la memoria allocata, identifiando le istruzioni illegali.

Non possiamo fare debug su librerie: occorre testare prima di creare la libreria.
Per abilitare il debugging, da terminale si fa cosi con "-g":

```
gcc -g -Wall -pedantic main.c -o main
gdb main
```

Normalmente, il debug si ferma alla prima istruzione eseguibile, ma possiamo modificare questo comportamento tramite:

- **Breakpoint:** un punto nel codice dove fermarsi
- **Watchpoint:** variabile da monitorare; il programma si ferma quando la variabile viene cambiata.

# 6 Esercizi

## 6.1 Esercizio 1

Partendo dalla propria home, svolgere queste operazioni dal terminale LINUX:

1. Creare una directory lezione02
2. Entrarci
3. Creare tre file vuoti: ”a.c”, ”b.c”, ”note.txt”
4. Visualizzare solamente i file con estensione ”.c”
5. Salvare quell’elenco in ”sorgenti.txt”
6. Aggiungere il contenuto di ”sorgenti.txt” a ”note.txt”
7. Contare quante righe contiene ”note.txt”

**Soluzione:**

```
$ cd ~
$ mkdir lezione02
$ cd lezione02
$ touch a.c b.c node.txt
$ ls *.c > sorgenti.txt
$ sorgenti.txt >> note.txt
$ qw -l note.txt
```